

Supplementary Material - Toy example inference codes

Marion Kerioui, Julie Bertrand, René Bruno,
François Mercier, Jérémie Guedj, Solène Desmée

October 2021

1 Joint_Monolix.txt

DESCRIPTION: CURRENT SLD

[LONGITUDINAL]

input = {BSLD, d, g, phi, lambda, beta}

EQUATION:

$$\begin{aligned} \text{SLD} &= \text{BSLD} * (\text{phi} * \exp(-d * (t)) + (1 - \text{phi}) * \exp(g * (t))) \\ \text{haz} &= (1 / \text{lambda}) * \exp(\text{beta} * \text{SLD}) \end{aligned}$$

DEFINITION:

survival = {type=event, hazard=haz}

OUTPUT:

output = {SLD, survival}

2 Joint_Monolix.mlxtran

<DATAFILE>

[FILEINFO]

file = 'Simulated_Dataset_Monolix.txt'

delimiter = comma

header = {ID, Time, Obs, Censor_SLD, Ytype}

[CONTENT]

ID = {use=identifier}

Time = {use=time}

Obs = {use=observation, name={y1, y2}, yname={'1', '2'}, type={continuous, event}}

Censor_SLD = {use=censored}

Ytype = {use=observationtype}

<MODEL>

[INDIVIDUAL]

input = {BSLD_pop, omega_BSLD, d_pop, omega_d, g_pop,
omega_g, phi_pop, omega_phi, lambda_pop, beta_pop}

DEFINITION:

BSLD = {distribution=logNormal, typical=BSLD_pop, sd=omega_BSLD}

d = {distribution=logNormal, typical=d_pop, sd=omega_d}

g = {distribution=logNormal, typical=g_pop, sd=omega_g}

phi = {distribution=logitNormal, typical=phi_pop, sd=omega_phi}

lambda = {distribution=logNormal, typical=lambda_pop, no-variability}

beta = {distribution=normal, typical=beta_pop, no-variability}

[LONGITUDINAL]

input = {b1}

file = 'Joint_Monolix.txt'

DEFINITION:

y1 = {distribution=normal, prediction=SLD, errorModel=proportional(b1)}

<FIT>

data = {y2, y1}

model = {survival, y1}

<PARAMETER>

BSLD_pop = {value=60, method=MLE}

b1 = {value=0.3, method=MLE}

beta_pop = {value=0.002, method=MLE}

```

d_pop = {value=0.0055, method=MLE}
g_pop = {value=0.0015, method=MLE}
lambda_pop = {value=1000, method=MLE}
omega_BSLD = {value=1, method=MLE}
omega_d = {value=1, method=MLE}
omega_g = {value=1, method=MLE}
omega_phi = {value=1, method=MLE}
phi_pop = {value=0.2, method=MLE}

```

<MONOLIX>

[TASKS]

```

populationParameters()
individualParameters(method = {conditionalMean, conditionalMode })
fim(method = StochasticApproximation)
logLikelihood(method = ImportanceSampling)
plotResult(method = {outputplot, indfits, obspred, parameterdistribution,
covariatemodeldiagnosis, randomeffects, covariancemodeldiagnosis, saemresults })

```

[SETTINGS]

GLOBAL:

```

exportpath = 'running'

```

3 Joint_stan.stan

```
//block defining functions
functions{

  /*Mechanistic model for tumor size kinetics for treatment initiation at t=0*/
  real SLD(real t, row_vector eta, vector mu, int nb_param){
    vector[nb_param] psi;
    real sld;

    for(j in 1:(nb_param-1)){
      psi[j]=exp(log(mu[j])+eta[j]);
    }

    psi[nb_param]=inv_logit(logit(mu[nb_param])+eta[nb_param]);
    sld=psi[1]*(psi[4]*exp(-psi[2]*t)+(1-psi[4])*exp(psi[3]*t));

    return sld;
  }

  /*Baseline exponential hazard function*/
  real h0(real lambda){
    real hr;
    hr=(1/lambda);
    return(hr);
  }

  /*Hazard function depending on the tumor size kinetics
  defined as an integrable function */
  real[] h_int(real t,real[] y, real[] theta, real[] x_r, int[] x_i ){
    real SLDtemp;
    real h;
    real dSdt[1];
    real lambda=theta[1];
    real beta=theta[2];
    int nb_param=x_i[1];

    vector[nb_param] mu;
    row_vector[nb_param] eta;
```

```

    mu[1]=theta[3];
    mu[2]=theta[4];
    mu[3]=theta[5];
    mu[4]=theta[6];

    eta[1]=theta[7];
    eta[2]=theta[8];
    eta[3]=theta[9];
    eta[4]=theta[10];

    h=h0(lambda);
    SLDtemp=fmin(SLD(t,eta, mu, nb_param),8000);
    //minimum between the SLD and 8000 to avoid potential numerical issues
    dSdt[1]=h*exp(beta*SLDtemp);//hazard function depending on SLD

    return(dSdt);
}

}

data{
    //longitudinal data
    int nb_obs;
    int<lower=1> N;
    int nb_param;
    int nb_time[N];
    int<lower=0> nb_max_time;
    vector[nb_obs] measures;
    vector[nb_obs] censor;
    real c;
    vector[nb_max_time] day;

    //survival data
    real T[N,1];
    vector[N] delta;

    //integration parameters
    real t_0;
    real y_0[1];

}

```

```

transformed data{
  //integration parameters
  real x_r[0];
  real y[0];
}

parameters{
  //population parameters
  real <lower=0> mu_1;//mu_BSLD
  real <lower=0> mu_2;//mu_d
  real <lower=0> mu_3;//mu_g
  real <lower=0, upper=1> mu_4;//mu_phi

  vector<lower=0>[nb_param] Omega;//Omega vector
  real<lower=0> sigma;//Standard deviation for error process
  real<lower=0> lambda;//exponential survival parameter
  real beta;//link parameter

  //random effects
  matrix[N,nb_param] eta_tilde;
}

transformed parameters{
  vector[nb_param] mu;
  //Non-Centered reparametrization for hierarchical models
  matrix[N,nb_param] eta;
  for(j in 1:nb_param){
    eta[,j]=eta_tilde[,j]*Omega[j];
  }

  mu[1]=mu_1;
  mu[2]=mu_2;
  mu[3]=mu_3;
  mu[4]=mu_4;
}

model{
  int w;
  real theta[10];
  real SLDtemp;

```

```

int xi[1];
xi[1]=nb_param;
w=0;

//population parameters
theta[1]=lambda;
theta[2]=beta;
theta[3]=mu[1];
theta[4]=mu[2];
theta[5]=mu[3];
theta[6]=mu[4];

/*Model log-likelihood*/
for(i in 1:N){
  //random effects of patient i
  theta[7]=eta[i,1];
  theta[8]=eta[i,2];
  theta[9]=eta[i,3];
  theta[10]=eta[i,4];

  //each time for each patient
  for(t in 1:nb_time[i]){
    SLDtemp=SLD(day[t], eta[i,], mu, nb_param);
    if(censor[w+t]==0){measures[w+t]~normal(SLDtemp, SLDtemp*sigma);}
  //density of longitudinal process
    if(censor[w+t]==1){target+=normal_lcdf(c| SLDtemp, SLDtemp*sigma);}
  //density of longitudinal process in case of censoring (LoQ, c=2.5mm)
  }

  w=w+nb_time[i];
  //prior on random effects
  eta_tilde[i,]~normal(0,1);

  if(delta[i]==1){//density of survival process in case of death
    target+=log(h_int(T[i,1], y, theta,x_r,xi)[1]
    *exp(-integrate_ode_rk45(h_int,y_0, t_0, T[i,], theta,x_r,xi)[1,1]));
  }

  if(delta[i]==0){//density of survival process in case of censoring
    target+=log(exp(-integrate_ode_rk45(h_int,y_0, t_0, T[i,], theta,x_r,xi)[1,1]));
  }
}

```

```

}

/*priors definition*/
mu_1~lognormal(4.8,1);
mu_2~beta(1,100);
mu_3~beta(1,100);
mu_4~beta(2,4);
Omega~lognormal(0,1);
sigma~lognormal(0,1);
lambda~lognormal(5,1);
beta~normal(0,0.5);
}

```

4 Launch_stan.R

```
#####
##### Stan inference for nonlinear joint model #####
#####

#####
##### Modelling the association between biomarkers and clinical
##### outcome: an introduction to nonlinear joint models
#####

### Marion Kerioui
### Julie Bertrand
### René Bruno
### François Mercier
### Jérémie Guedj
### Solène Desmée

rm(list=ls())
setwd(dir="")#path to working directory
library(rstan)#loading rstan library
rstan_options(auto_write = TRUE)

####Example of a simulated Dataset-----
data=read.table("Simulated_Dataset.txt", sep=",", header=TRUE)

####Data description----
summary(data)
unique(data$ID) #identifiers of the 100 patients
data$Time #time of SLD measurements
data$SLD #measurements of SLD
data$Censor_SLD #indicator of left-censoring for SLD below the LoQ
data[which(data$Time==0),]$T #Time of event (death or censor)
data[which(data$Time==0),]$delta #event indicator (=1 if death)

####Formatting Data----
nb_obs=length(data$SLD)#total number of observations
N=length(unique(data$ID))#number of patients

nb_time=c()#number of SLD measurements per patient
for(i in 1:N){
  nb_time[i]=length(data[which(data$ID==i),]$Time)
}
nb_max_time=max(nb_time)#maximum number of SLD measurements
```

```

measures=data$SLD#SLD measurements
censor=data$Censor_SLD#indicator of censoring

day=seq(0,693,63)#time of SLD measurements
T=data[which(data$Time==0),]$T #time of event
T=matrix(nrow=N, ncol=1, T) #time of event in a matrix

delta=data[which(data$Time==0),]$delta #indicator of event
nb_param=4#number of parameters

###integration parameters
t_0=0
y_0=array(0)

#####list of data given to stan function----
dat=list(nb_obs=nb_obs, N=N, nb_time=nb_time, nb_max_time=nb_max_time,
measures=measures,censor=censor,day=day, T=T, delta=delta, t_0=t_0,
y_0=y_0,nb_param=nb_param, c=2.5 )

####Parametrization of HMC algorithm-----
nb_iter=400 #number of iterations
nb_warmup=200#number of warm up iterations
nb_chains=3#number of chains
nb_cores=nb_chains #one chain per core

####control parameters
treedepth=12
adapt_delta=0.9

####Initialisation of population parameters and random effects
inits=list(mu_1=60,mu_2=0.0055, mu_3=0.0015, mu_4=0.2, sigma=0.18,
Omega=c(0.7,1,1,1.5),lambda=580,beta=0,
eta=matrix(rep(0,N*nb_param), N, nb_param))

####Run NUTS-HMC----
fit=stan(file="Joint_stan.stan",data=dat,iter=nb_iter,
        warmup=nb_warmup, chains=nb_chains,
init = list(inits,inits,inits),cores=nb_cores,
control=list(max_treedepth=treedepth, adapt_delta=adapt_delta))

####Results-----
summary(fit)#summary of the 3 HMC chains
stan_trace(fit)#trace plots

```

